

Before running any program

In PyCharm Terminal:

```
pip install selenium
```

Make sure **Google Chrome** is installed. Recent Selenium versions normally manage the required ChromeDriver automatically.

1. Open Website and Verify Title

Concept: Basic browser automation + title verification.

Code

```
from selenium import webdriver
import time
```

```
# Create Chrome browser
driver = webdriver.Chrome()
```

```
# Open Google
driver.get("https://www.google.com")
```

```
# Maximize browser
driver.maximize_window()
```

```
# Wait for page to load
time.sleep(2)
```

```
# Get page title
title = driver.title
```

```
print("Page Title:", title)
```

```
# Verify title
if "Google" in title:
    print("Test Passed")
else:
    print("Test Failed")
```

```
# Close browser
driver.quit()
```

Output

```
Page Title: Google
Test Passed
```

Line-by-line explanation

Code	Explanation
<code>from selenium import webdriver</code>	Imports Selenium WebDriver.
<code>driver = webdriver.Chrome()</code>	Opens Chrome browser.
<code>driver.get(...)</code>	Opens the specified website.
<code>driver.maximize_window()</code>	Maximizes the browser.
<code>time.sleep(2)</code>	Waits 2 seconds.
<code>driver.title</code>	Gets the webpage title.
<code>print()</code>	Displays the title.
<code>if "Google" in title</code>	Checks whether Google exists in the title.
<code>driver.quit()</code>	Closes the browser.

2. Search Text in Google

Concept: Finding a web element using `name`.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
```

```
driver = webdriver.Chrome()
```

```
driver.get("https://www.google.com")
driver.maximize_window()
```

```
time.sleep(2)
```

```
# Find Google search box
search_box = driver.find_element(By.NAME, "q")
```

```
# Enter search text
search_box.send_keys("Selenium Python")
```

```
# Press Enter
search_box.submit()
```

```
time.sleep(3)
```

```
print("Search completed successfully")
print("Page Title:", driver.title)
```

```
driver.quit()
```

Output

Search completed successfully

Page Title: Selenium Python - Google Search

Important Selenium commands

```
find_element()
```

Finds an element.

```
send_keys()
```

Types text into an element.

```
submit()
```

Submits a form.

3. Find Element Using ID

Concept: Locate an HTML element using **By.ID**.

For a student demonstration, we can use a simple online page specifically designed for Selenium practice.

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
import time
```

```
driver = webdriver.Chrome()
```

```
driver.get("https://the-internet.herokuapp.com/login")
```

```
driver.maximize_window()
```

```
time.sleep(2)
```

```
# Find username field using ID
```

```
username = driver.find_element(By.ID, "username")
```

```
# Enter username
```

```
username.send_keys("tomsmith")
```

```
# Find password field
```

```
password = driver.find_element(By.ID, "password")
```

```
# Enter password
```

```
password.send_keys("SuperSecretPassword!")
```

```
# Find login button
```

```
login_button = driver.find_element(By.CSS_SELECTOR, "button[type='submit']")

# Click login
login_button.click()

time.sleep(2)

print("Current URL:", driver.current_url)

driver.quit()
```

Output

Current URL: https://the-internet.herokuapp.com/secure

Main concepts

```
By.ID
By.CSS_SELECTOR
send_keys()
click()
current_url
```

4. Login Automation – PASS Test

Concept: Automating a login form.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

try:
    driver.get("https://the-internet.herokuapp.com/login")
    driver.maximize_window()

    time.sleep(2)

    username = driver.find_element(By.ID, "username")
    password = driver.find_element(By.ID, "password")
    login_button = driver.find_element(
        By.CSS_SELECTOR,
        "button[type='submit']"
    )

    username.send_keys("tomsmith")
    password.send_keys("SuperSecretPassword!")
```

```

login_button.click()

time.sleep(2)

if "secure" in driver.current_url:
    print("Login Test Passed")
else:
    print("Login Test Failed")

finally:
    driver.quit()

```

Output

Login Test Passed

Why **try...finally**?

```

try:
    Contains the automation steps.
finally:
    driver.quit()

```

Ensures that Chrome closes even if an error occurs.

5. Login Automation – FAIL Test Case

This is useful for teaching students **positive and negative testing**.

```

from selenium import webdriver
from selenium.webdriver.common.by import By
import time

```

```

driver = webdriver.Chrome()

```

```

try:
    driver.get("https://the-internet.herokuapp.com/login")
    driver.maximize_window()

```

```

time.sleep(2)

```

```

username = driver.find_element(By.ID, "username")
password = driver.find_element(By.ID, "password")
login_button = driver.find_element(
    By.CSS_SELECTOR,
    "button[type='submit']"
)

```

```

username.send_keys("tomsmith")

```

```

# Wrong password intentionally
password.send_keys("WrongPassword")

login_button.click()

time.sleep(2)

if "secure" in driver.current_url:
    print("Test Passed")
else:
    print("Test Failed: Invalid login was correctly rejected")

finally:
    driver.quit()

```

Output

Test Failed: Invalid login was correctly rejected

This is a useful classroom discussion:

Technically, the application behaved correctly, but our simple `if` statement calls it "Test Failed."

A better assertion-based version is shown later.

6. Click a Link

Concept: Link automation using `By.LINK_TEXT`.

```

from selenium import webdriver
from selenium.webdriver.common.by import By
import time

```

```

driver = webdriver.Chrome()

```

```

driver.get("https://the-internet.herokuapp.com/")
driver.maximize_window()

```

```

time.sleep(2)

```

```

# Click "Form Authentication"

```

```

link = driver.find_element(By.LINK_TEXT, "Form Authentication")

```

```

link.click()

```

```

time.sleep(2)

```

```

print("Current Page:", driver.title)
print("Current URL:", driver.current_url)

```

```
driver.quit()
```

Output

Current Page: The Internet

Current URL: <https://the-internet.herokuapp.com/login>

Important

By.LINK_TEXT

is used when we know the exact visible text of a link.

7. Dropdown Automation

Concept: Selecting an option from a dropdown.

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.support.ui import Select
```

```
import time
```

```
driver = webdriver.Chrome()
```

```
driver.get("https://the-internet.herokuapp.com/dropdown")
```

```
driver.maximize_window()
```

```
time.sleep(2)
```

```
# Locate dropdown
```

```
dropdown = driver.find_element(By.ID, "dropdown")
```

```
# Create Select object
```

```
select = Select(dropdown)
```

```
# Select Option 2
```

```
select.select_by_visible_text("Option 2")
```

```
time.sleep(2)
```

```
# Print selected option
```

```
selected = select.first_selected_option.text
```

```
print("Selected Option:", selected)
```

```
driver.quit()
```

Output

Selected Option: Option 2

Important Selenium class

Select

is used for HTML `<select>` dropdowns.

8. Checkbox Automation

Concept: Selecting and unselecting checkboxes.

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
import time
```

```
driver = webdriver.Chrome()
```

```
driver.get("https://the-internet.herokuapp.com/checkboxes")
```

```
driver.maximize_window()
```

```
time.sleep(2)
```

```
# Find all checkboxes
```

```
checkboxes = driver.find_elements(By.CSS_SELECTOR, "input[type='checkbox']")
```

```
print("Number of checkboxes:", len(checkboxes))
```

```
# Select first checkbox if not selected
```

```
if not checkboxes[0].is_selected():
```

```
    checkboxes[0].click()
```

```
print("First checkbox selected:",
```

```
      checkboxes[0].is_selected())
```

```
# Select second checkbox if not selected
```

```
if not checkboxes[1].is_selected():
```

```
    checkboxes[1].click()
```

```
print("Second checkbox selected:",
```

```
      checkboxes[1].is_selected())
```

```
driver.quit()
```

Output

Number of checkboxes: 2

First checkbox selected: True

Second checkbox selected: True

Important methods

`find_elements()`

Finds multiple elements.


```
is_selected()
Checks whether checkbox is selected.
```

9. Screenshot Automation

Concept: Taking a screenshot during testing.

```
from selenium import webdriver
import time
```

```
driver = webdriver.Chrome()
```

```
driver.get("https://www.google.com")
driver.maximize_window()
```

```
time.sleep(2)
```

```
# Take screenshot
driver.save_screenshot("google_homepage.png")
```

```
print("Screenshot saved successfully")
```

```
driver.quit()
```

Output

Screenshot saved successfully

The screenshot will be saved in your project's working directory as:
google_homepage.png

10. Selenium + **unittest** – PASS and FAIL

This is especially useful for your **unit testing practical**.

```
import unittest
from selenium import webdriver
```

```
class TestGoogle(unittest.TestCase):
```

```
    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
```

```
    def test_google_title_pass(self):
        self.driver.get("https://www.google.com")
```

```
        title = self.driver.title
```

```

print("Actual Title:", title)

self.assertIn("Google", title)

def test_google_title_fail(self):
    self.driver.get("https://www.google.com")

    title = self.driver.title

    print("Actual Title:", title)

    # Intentionally wrong expected value
    self.assertEqual(title, "Facebook")

def tearDown(self):
    self.driver.quit()

if __name__ == "__main__":
    unittest.main()

```

Expected output

Actual Title: Google
Actual Title: Google

Ran 2 tests

FAILED (failures=1)

PyCharm will show:

test_google_title_pass PASSED

test_google_title_fail FAILED

The failure is intentional.

Selenium Locators Students Should Learn

These are the most important locator types for your practicals:

Locator	Syntax	Example
ID	By.ID	By.ID, "username"
Name	By.NAME	By.NAME, "q"
Class	By.CLASS_NAME	By.CLASS_NAME, "btn"

Tag	By.TAG_NAME	By.TAG_NAME, "input"
Link Text	By.LINK_TEXT	By.LINK_TEXT, "Login"
Partial Link	By.PARTIAL_LINK_TEXT	By.PARTIAL_LINK_TEXT, "Log"
CSS	By.CSS_SELECTOR	By.CSS_SELECTOR, "#username"
XPath	By.XPATH	By.XPATH, "//input[@id='username']"

Selenium actions students should practice

get()
 find_element()
 find_elements()
 click()
 send_keys()
 clear()
 submit()
 get_attribute()
 is_displayed()
 is_enabled()
 is_selected()
 current_url
 title
 save_screenshot()
 quit()

Recommended practical sequence

For students, I would teach Selenium in this order:

Practical 1: Open browser and website

↓

Practical 2: Verify webpage title

↓

Practical 3: Locate elements

↓

Practical 4: Enter text using `send_keys()`

↓

Practical 5: Click buttons/links

↓

Practical 6: Automate login

↓

Practical 7: Dropdown

↓

Practical 8: Checkbox

↓

Practical 9: Screenshot

↓

Practical 10: Assertions

↓

Practical 11: `unittest` with Selenium

↓

Practical 12: PASS/FAIL test cases

↓

Practical 13: Explicit waits

↓

Practical 14: Mouse and keyboard actions

↓

Practical 15: Complete mini Selenium automation project